

GUI-Based Multi-Client Chat Application Using TCP

Assignment 6 — Networking Internship, ISEA Phase 3
Tezpur University

Name	Madhab Boro
Roll No.	CSB24055
Repository	ISEA-Phase3-TezpurUniversity- GUIChatApplication

1. Objective

The objective of this assignment is to convert the terminal-based TCP chat application developed in Assignment 5 into a graphical desktop application while reusing the existing server implementation without modification to its core logic. The project demonstrates the integration of GUI programming with socket-based network programming and highlights the following learning outcomes:

- Building an event-driven graphical user interface using Tkinter.
- Establishing reliable client-server communication using TCP sockets.
- Applying multithreading to keep the GUI responsive while messages are received in the background.
- Reusing and extending an existing client-server architecture rather than rebuilding it from scratch.
- Supporting real-time broadcast and private messaging between multiple clients over a Mininet network.

2. GUI Design

The client interface is implemented in **client_gui.py** using Tkinter and consists of two windows: a login window and a main chat window. The design keeps the interface minimal and functional, in line with the assignment's emphasis on functionality over visual complexity.

2.1 Login Window

The login window is the entry point of the application. It contains a username field, an optional password field, and a Connect button. Input validation prevents an empty username from being submitted, and a message box informs the user of successful or failed connection attempts before the main chat window is launched.

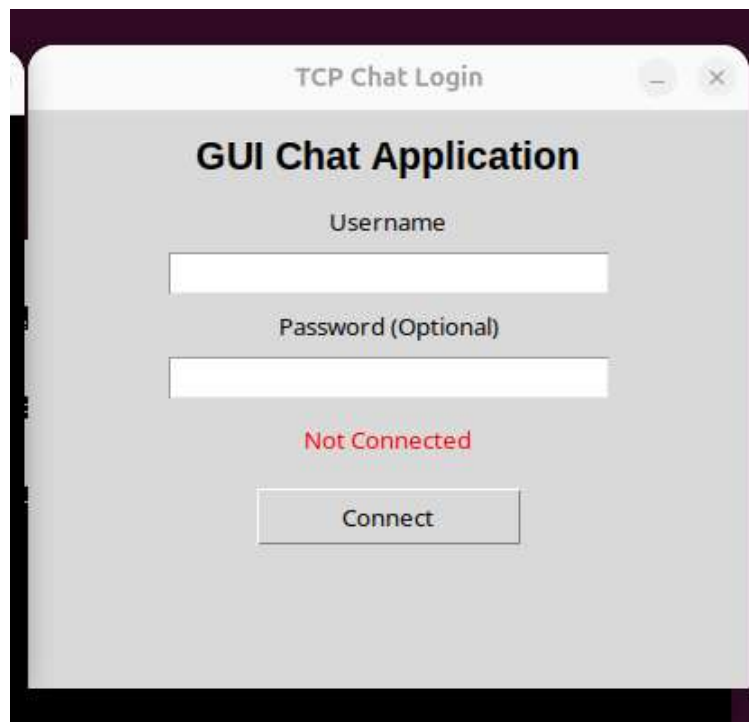


Figure 2.1 — Login window with username and optional password fields, and a "Not Connected" status indicator before the Connect button is pressed.

2.2 Main Chat Window

The chat window is organised using Tkinter **Frame** containers into three regions: a scrollable message display area on the left, an online users list on the right, and a bottom panel containing the message entry box, Send button, and Disconnect button. A status label at the bottom of the window indicates whether the client is currently connected or disconnected.

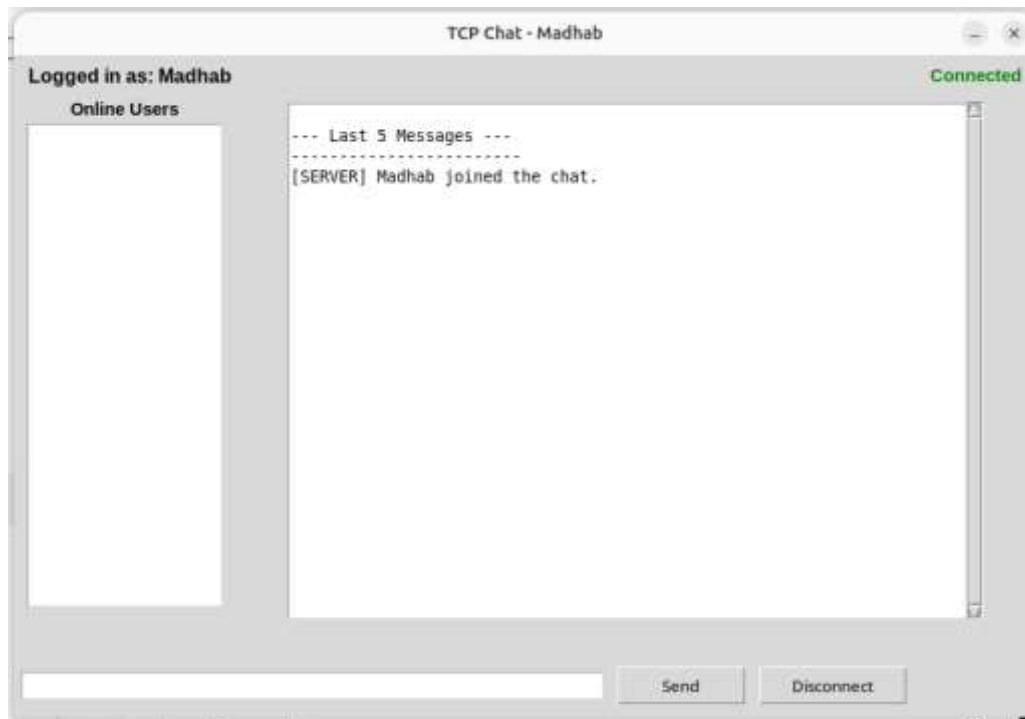


Figure 2.2 — Main chat window for user "Madhab" immediately after connecting, showing the message display area, online users panel, message entry box, and the "Connected" status indicator in green.

2.3 Widgets Used

Widget	Purpose
Label	Display field names, headings, and the connection status indicator
Entry	Username, password, and message input fields
Button	Connect, Send, and Disconnect actions
Frame	Structural grouping of the login form, chat area, and control panel
ScrolledText	Scrollable, auto-updating message display area
Listbox	Live list of currently online users
Messagebox	Pop-up dialogs for validation errors and connection feedback

3. System Architecture

The system follows a client-server architecture over TCP. The server (h1) runs on a fixed port and accepts connections from multiple clients (h2, h3, h4, h5) inside a Mininet single-switch topology of five hosts.

```
sudo mn --topo single,5
```

Connectivity between hosts was verified before running the application using the Mininet commands `nodes`, `net`, and `pingall`.

3.1 Topology

```
Mininet
h1
Chat Server
|
-----
| | | |
h2 h3 h4 h5
Client A Client B Client C Client D
```

3.2 Message Flow

Each client establishes a single persistent TCP connection to the server on port 5000. Two logical flows operate concurrently on the client side: the main thread handles GUI events (typing, clicking Send, clicking Disconnect), while a background thread continuously listens for incoming data from the server socket and forwards it to the GUI for display. On the server side, each client connection is handled by its own thread, and outgoing messages are broadcast to all connected clients or routed to a specific client for private messages.

3.3 Project Structure

```
server.py
client_gui.py
chat_history.csv
performance_results.csv
screenshots/
README.md
report.pdf
```

4. Components Reused from Assignment 5

As required by the assignment, the server implementation and the core networking logic were reused with minimal modification. The following components were carried over directly from the Assignment 5 terminal-based chat application:

- **server.py** — the multi-threaded TCP server, including client connection handling, message broadcasting, and private message routing, was reused unchanged.
- **Socket communication logic** — the TCP socket creation, connect/send/receive calls, and message framing were kept identical to Assignment 5.
- **Online user management** — the server-side logic that tracks connected clients and notifies all clients of joins and leaves was reused as-is.
- **Chat history and logging** — the mechanism for recording messages to `chat_history.csv` and performance statistics to `performance_results.csv` was retained from Assignment 5.
- **Broadcast and private messaging protocol** — the message format used to distinguish broadcast messages from private messages was preserved so that the new GUI client remains fully compatible with the existing server.

Only the client-side presentation layer was replaced: the terminal input/output loop of Assignment 5 was substituted with the Tkinter-based GUI in **client_gui.py**, while the underlying network calls remain

the same.

5. GUI Design Decisions

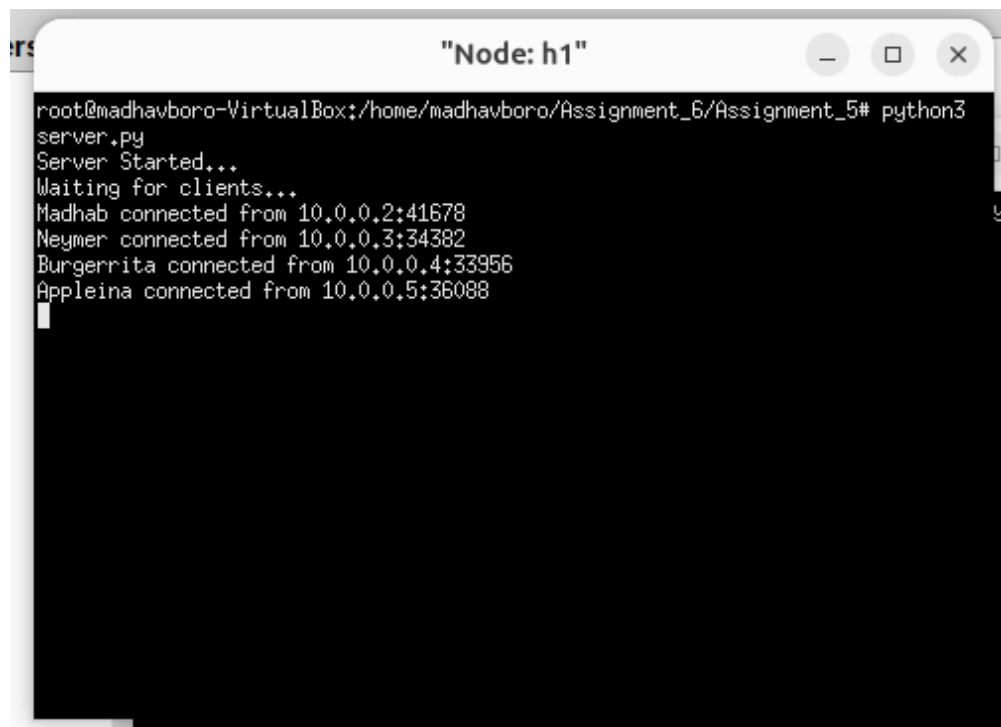
- **Separation of networking and GUI code:** all socket operations are kept in dedicated functions/classes separate from the Tkinter widget callbacks, so the interface layer can be modified independently of the communication logic.
- **Non-blocking design:** a background thread is used exclusively for `socket.recv()` calls so that the Tkinter main loop is never blocked and the interface remains responsive at all times.
- **Thread-safe GUI updates:** incoming messages received on the background thread are pushed to the ScrolledText widget and Listbox using thread-safe scheduling (`widget.after()`), avoiding direct cross-thread widget access.
- **Simplicity over decoration:** per the assignment's guidance to prioritise functionality, the layout uses plain frames and default styling rather than custom themes, keeping the codebase easy to read and extend.
- **Automatic scrolling:** the message display automatically scrolls to the latest message on arrival, so users do not need to manually scroll during an active conversation.
- **Immediate feedback:** the status label and message boxes give the user immediate, visible feedback for connection, disconnection, and validation errors.

6. Testing Results

The application was tested inside Mininet using the five-host topology (h1 as server, h2–h5 as clients). The following scenarios were executed and verified successfully:

Test Scenario	Result
Client login with valid username	Pass
Client login with empty username (validation)	Pass — error displayed, connection blocked
Multiple simultaneous client connections (4 clients)	Pass
Broadcast messaging to all connected clients	Pass
Private messaging between two selected clients	Pass
Join notification displayed to existing clients	Pass
Leave notification displayed to remaining clients	Pass
Online user list updates automatically	Pass
GUI remains responsive while messages arrive	Pass
Graceful disconnect via Disconnect button	Pass

The screenshots below, captured while testing with four Mininet clients (Madhab, Neymar, Burgerrita, and Appleina), document each verified scenario.



```
root@madhavboro-VirtualBox:/home/madhavboro/Assignment_6/Assignment_5# python3
server.py
Server Started...
Waiting for clients...
Madhab connected from 10.0.0.2:41678
Neymer connected from 10.0.0.3:34382
Burgerrita connected from 10.0.0.4:33956
Appleina connected from 10.0.0.5:36088
```

Figure 6.1 — Server terminal (host h1) showing the server starting up and accepting connections from all four clients, each identified by its username and Mininet IP address.

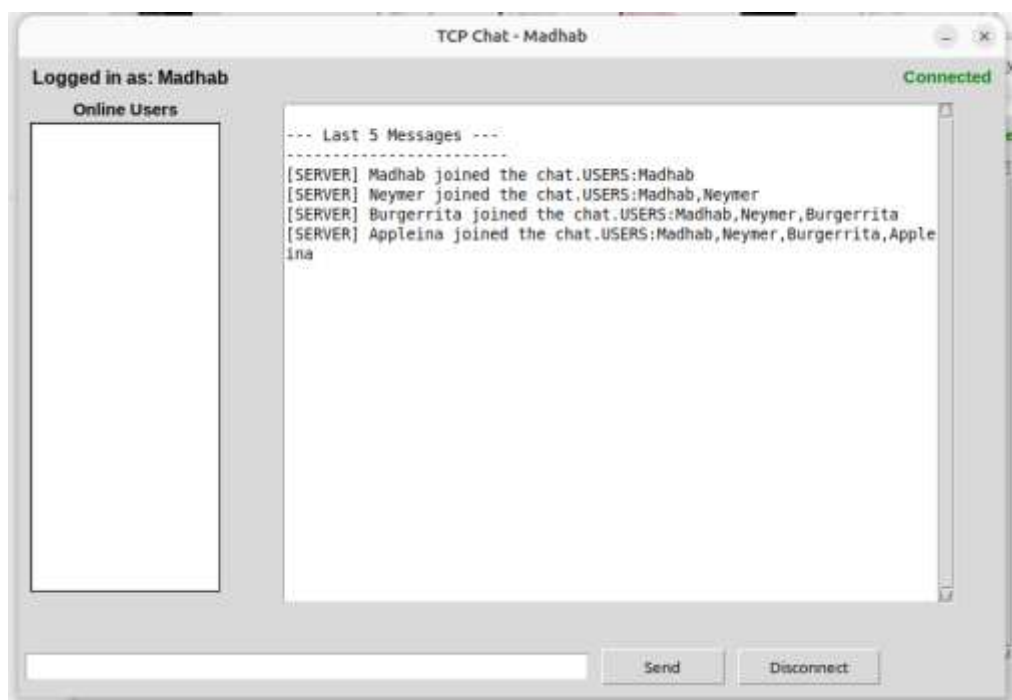


Figure 6.2 — Chat window showing join notifications as Madhab, Neymer, Burgerrita, and Appleina connect one after another, with the online users list updating for each new client.

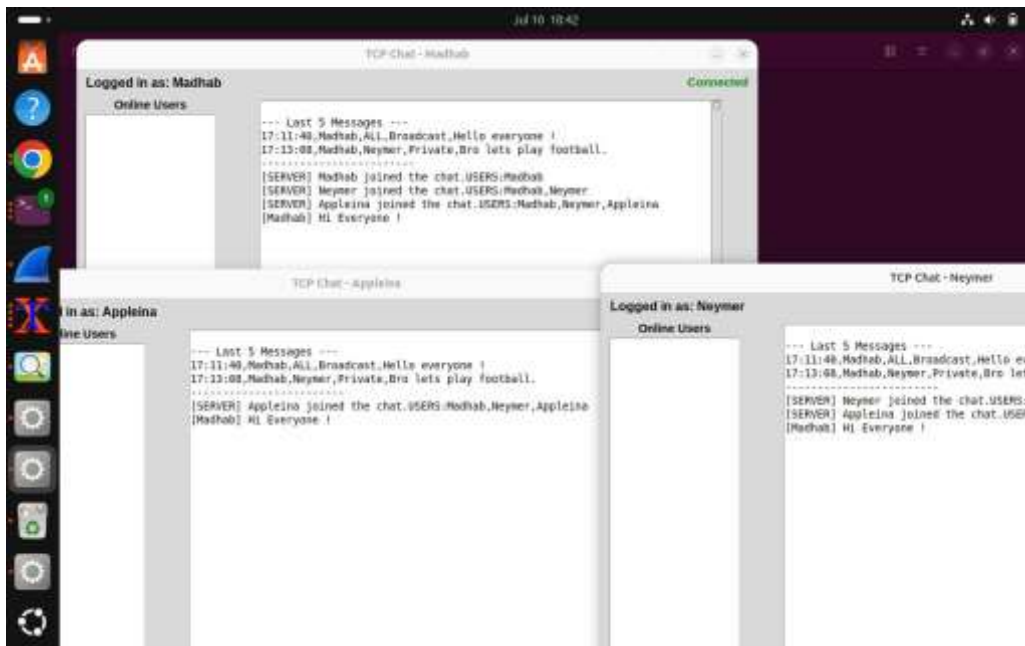


Figure 6.3 — A broadcast message sent by Madhab ("Hi Everyone!") appearing simultaneously in the chat windows of Madhab, Appleina, and Neymar, confirming successful broadcast delivery to all connected clients.



Figure 6.4 — A private message from Madhab to Neymar ("Bro lets play football.") shown as [To Neymar] in the sender's window and as [PRIVATE] in the recipient's window, confirming the message was not broadcast to other clients.

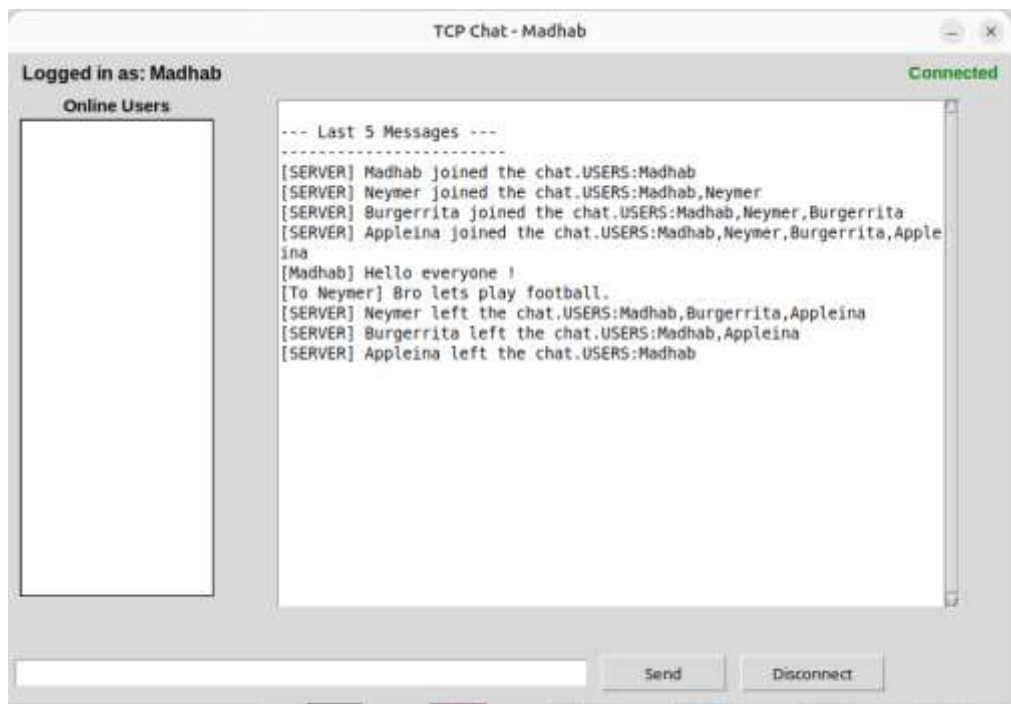


Figure 6.5 — Chat window showing leave notifications as Neymar, Burgerrita, and Appleina disconnect in sequence, with the server-broadcast "left the chat" messages and the updated user list after each departure.

7. Wireshark Verification

Network traffic between the server and clients was captured using Wireshark with the following display filter:

```
tcp.port == 5000
```

The capture was used to verify the following at the packet level:

- **TCP three-way handshake** — SYN, SYN-ACK, and ACK packets confirming establishment of each client connection to the server.
- **Broadcast message transmission** — a single message sent from one client observed being relayed by the server to all other connected clients as separate TCP segments.
- **Private message transmission** — a message directed at a specific client observed being delivered only to the intended recipient's connection.
- **Connection termination** — FIN/ACK exchange confirming graceful closure of the TCP connection when a client disconnects.



Figure 7.1 — TCP three-way handshake (SYN, SYN-ACK, ACK) between client 10.0.0.2 (port 41678) and the server 10.0.0.1 (port 5000), confirming successful establishment of the client connection.



Figure 7.2 shows a Wireshark packet capture of a broadcast message from the server (10.0.0.1, port 5000) to multiple clients. The filter is 'tcp.port == 5000'. The table below represents the data visible in the packet list:

No.	Time	Source	Destination	Protocol	Length	Info
182	443.724322676	10.0.0.1	10.0.0.5	TCP	60	5000 → 36088 [ACK] Seq=1 Ack=9 Win=43520 Len=0 TSval=...
183	443.726322018	10.0.0.1	10.0.0.5	TCP	60	5000 → 36088 [PSH, ACK] Seq=1 Ack=9 Win=43520 Len=25
184	443.726330674	10.0.0.1	10.0.0.3	TCP	60	5000 → 5000 [ACK] Seq=9 Ack=26 Win=42496 Len=0 TSval=...
185	443.726340508	10.0.0.1	10.0.0.4	TCP	60	5000 → 5000 [PSH, ACK] Seq=9 Ack=26 Win=42496 Len=25
186	443.726347625	10.0.0.1	10.0.0.2	TCP	60	5000 → 5000 [ACK] Seq=9 Ack=26 Win=42496 Len=0 TSval=...
187	443.726356882	10.0.0.1	10.0.0.2	TCP	100	5000 → 41678 [PSH, ACK] Seq=212 Ack=7 Win=43520 Len=3
188	443.726370258	10.0.0.1	10.0.0.3	TCP	100	5000 → 34382 [PSH, ACK] Seq=228 Ack=7 Win=43520 Len=3
189	443.726379474	10.0.0.1	10.0.0.4	TCP	100	5000 → 33956 [PSH, ACK] Seq=217 Ack=11 Win=43520 Len=3
190	443.726379383	10.0.0.1	10.0.0.5	TCP	100	5000 → 36088 [PSH, ACK] Seq=217 Ack=9 Win=43520 Len=3

Figure 7.2 — PSH, ACK segments from the server (10.0.0.1, port 5000) sent to multiple client ports (41678, 34382, 33956, 36088) at nearly the same timestamp, confirming the server relays a broadcast message to every connected client.

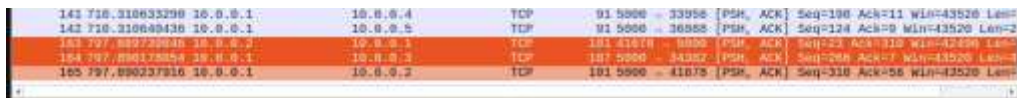


Figure 7.3 shows a Wireshark packet capture highlighting a private message from the server (10.0.0.1) to a single client (10.0.0.2, port 41678). The filter is 'tcp.port == 5000 && tcp.flags==F'. The table below represents the data visible in the packet list:

No.	Time	Source	Destination	Protocol	Length	Info
143	710.310633298	10.0.0.1	10.0.0.4	TCP	91	5000 → 33956 [PSH, ACK] Seq=190 Ack=11 Win=43520 Len=2
144	710.310640438	10.0.0.1	10.0.0.5	TCP	91	5000 → 36088 [PSH, ACK] Seq=124 Ack=9 Win=43520 Len=2
145	797.890720046	10.0.0.2	10.0.0.1	TCP	101	41678 → 5000 [PSH, ACK] Seq=23 Ack=212 Win=42496 Len=4
146	797.890728854	10.0.0.1	10.0.0.3	TCP	101	5000 → 34382 [PSH, ACK] Seq=228 Ack=7 Win=43520 Len=3
147	797.890737916	10.0.0.1	10.0.0.2	TCP	101	5000 → 41678 [PSH, ACK] Seq=210 Ack=55 Win=43520 Len=3

Figure 7.3 — PSH, ACK segments highlighting a message delivered from the server to a single client port (10.0.0.1 → 10.0.0.2, port 41678), confirming that a private message is routed only to the intended recipient's connection rather than broadcast to all clients.

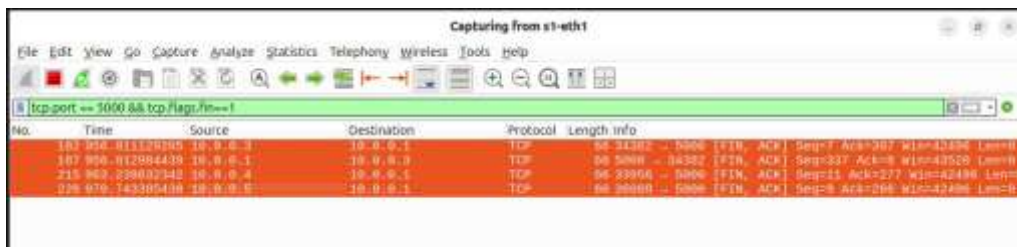


Figure 7.4 shows a Wireshark packet capture of FIN, ACK segments exchanged between the server (10.0.0.1) and clients (10.0.0.3, 10.0.0.4, 10.0.0.5) on port 5000. The filter is 'tcp.port == 5000 && tcp.flags==F'. The table below represents the data visible in the packet list:

No.	Time	Source	Destination	Protocol	Length	Info
187	898.811381949	10.0.0.3	10.0.0.1	TCP	60	5000 → 5000 [FIN, ACK] Seq=7 Ack=287 Win=42496 Len=0
187	898.812984438	10.0.0.4	10.0.0.1	TCP	60	5000 → 5000 [FIN, ACK] Seq=337 Ack=8 Win=43520 Len=0
215	962.2388027342	10.0.0.4	10.0.0.1	TCP	60	5000 → 5000 [FIN, ACK] Seq=11 Ack=277 Win=42496 Len=0
228	978.743305418	10.0.0.5	10.0.0.1	TCP	60	5000 → 5000 [FIN, ACK] Seq=28 Ack=266 Win=42496 Len=0

Figure 7.4 — FIN, ACK segments exchanged between the server (10.0.0.1) and each client (10.0.0.3, 10.0.0.4, 10.0.0.5) on port 5000, confirming graceful termination of the TCP connection as each client disconnects.

8. Reflection Answers

Q1. Why should networking logic and GUI code be separated?

Separating networking logic from GUI code improves maintainability, testability, and reusability. It allows the socket communication layer to be modified, debugged, or reused (for example, in a different interface such as a web front-end) without touching the widget layout, and vice versa. It also reduces the risk of blocking the interface, since networking code that is isolated from GUI callbacks can be safely run in a background thread.

Q2. Why is a background thread required?

Tkinter runs a single main event loop that is responsible for drawing the interface and responding to user actions such as clicks and key presses. Socket operations like `recv()` are blocking calls — they pause execution until data arrives. If this call were made on the main thread, the entire GUI would freeze whenever the client was waiting for a message. Running the receive loop on a separate background thread ensures the main thread stays free to process user interaction, keeping the interface responsive at all times.

Q3. Which components from Assignment 5 were reused?

The complete server implementation, the TCP socket communication logic, the online user management, the broadcast and private messaging protocol, and the chat history/logging mechanism were reused directly from Assignment 5, as detailed in Section 4 of this report. Only the client's terminal input/output loop was replaced with the Tkinter GUI.

Q4. How did the GUI improve usability?

The graphical interface removed the need for users to remember or type command syntax for actions such as sending a private message. A visible online users list makes it clear who is available to chat with, a scrollable, auto-updating message area removes the need to manually track conversation history, and dedicated Send/Disconnect buttons make common actions immediately discoverable. Overall, the GUI makes the application accessible to users who are not comfortable with a command-line interface.

Q5. Suggested future enhancement

A useful future enhancement would be to add end-to-end encryption of messages (for example using TLS or an application-level symmetric encryption scheme), so that message content captured in a Wireshark trace or intercepted on the network cannot be read in plain text. Other possible enhancements include file transfer support, message delivery/read receipts, and persistent user authentication.

9. Conclusion

This project successfully converted the terminal-based TCP chat application from Assignment 5 into a graphical desktop application using Tkinter, while reusing the server and core networking logic without modification. The resulting application supports GUI-based login, real-time broadcast and private messaging, automatic online-user tracking, and a responsive interface achieved through background-thread message handling. Testing across multiple Mininet clients and packet-level verification with Wireshark confirmed that the application behaves correctly under the required scenarios. The assignment reinforced key concepts in GUI programming, event-driven design, multithreading, and the importance of a clean separation between networking and presentation logic in a client-server application.

End of Report